

Proyecto Final: Estrategia de Trading Algorítmico

Profesor: Thomas Mac Millan

Ayudante: Nicolás Zanni (nzanni@uc.cl)

Objetivo

El objetivo de este proyecto es que desarrollen e implementen una estrategia de trading algorítmico. Se les dará la libertad de crear cualquier estrategia que deseen, ya sea de Arbitraje Estadístico, Momentum, Arbitraje, Factores, etc.

Dataset

Se les entregará un dataset en formato parquet, el cual tendrán que trabajarlo como un pandas dataframe. Contiene todas las transacciones de varios símbolos que tranzan en el exchange NYSE. En particular, cada fila contiene el símbolo, precio de la transacción, cantidad transada, y las mejores puntas justo antes del trade (bid_px_00 y ask_px_00). Este es un subset por columnas del schema TBBO de databento (<https://databento.com/docs/schemas-and-data-formats/tbbo#fields-tbbo?historical=python&live=python&reference=python>).

Para no caer en overfitting, se les brindará un dataset (28/3/23-29/12/23), en el cual podrán analizar datos, crear la estrategia y optimizarla, pero al igual que en la práctica, no nos interesa crear la mejor estrategia en el pasado, sino que queremos que sea rentable en el futuro. Por esto mismo, la evaluación de este proyecto se realizará en otro dataset con los mismo símbolos, pero en otro periodo, Si la estrategia es robusta, debería tener un comportamiento similar, al menos en ciertos periodos. Similarmente, para no estar sesgado por el nombre de los símbolos, estos se modificarán con letras (Símbolo A, B, ...).

Estos instrumentos financieros poseen cierta estructura de correlación, sectorial, o de país, las cuales podrán aprovechar para crear una estrategia rentable aunque igualmente, pueden tradeear cada símbolo de forma independiente. Como recomendación, analizen los datos, ploteenlos, usen herramientas de estadística, machine learning o lo que estimen conveniente, y en base a ello, sugieran alguna hipótesis de comportamiento entre algunos activos y luego prueben su idea con código.

Si agrupan por símbolo y por día, deberían ver esta estructura de retornos acumulados:



Código

La carpeta contendrá 4 archivos:

- *utils.py*: Archivo que contiene 2 clases fundamentales. Base Strategy, clase que contiene los métodos fundamentales para el desarrollo de su estrategia(explicados en el código) y de la cual va a heredar la clase Strategy que tendrán que implementar. La otra clase es TradeMonitor, de la cual podrán ver todas sus transacciones. Con el método show, podrán verlo en un formato más agradable dependiendo del lado de la operación. Un ejemplo sería:

time	symbol	exchange	price	qty	side
2024-09-09 10:28:19.661000	A	NYSE	100.500000	400	BUY
2024-09-09 10:29:44.124000	A	NYSE	101.500000	400	SELL
2024-09-09 10:35:12.453000	B	NYSE	1201.500000	30	SELL
2024-09-09 10:37:35.973000	B	NYSE	1201.500000	30	SELL
2024-09-09 10:38:51.682000	B	NYSE	1201.500000	5	SELL
2024-09-09 10:59:48.764000	A	NYSE	100.000000	500	BUY
2024-09-09 11:00:11.176000	A	NYSE	100.000000	200	BUY
2024-09-09 11:03:59.544000	A	NYSE	101.000000	200	SELL

- *analysis.ipynb*: Jupyter Notebook en donde crearan su estrategia, analizaran los datos, etc. El desarrollo de una estrategia de trading algorítmico es algo iterativo, con muchos aspectos que se van descubriendo a través del análisis de los datos y prueba y error. Por esto mismo, se tendrá en consideración para la evaluación este jupyter y su desarrollo. Idealmente, deberían complementar su código con markdowns para explicar el por qué de su código.
- *data.parquet*: El dataset en formato parquet. Este lo deben importar usando `pd.read_parquet('data.parquet')` donde lo necesiten.
- *main.py*: Al final de todo el proceso, la estrategia la deben copiar y pegar en este archivo. Es la clase Strategy de este archivo la cual se evaluará en el dataset nuevo.

En particular, deberán modificar solo el método `before_open` y `trading_session` de la clase `Strategy`. En estos pueden hacer lo que ustedes deseen, pero asegurándose de que cada vez que realicen un trade, lo hagan llamando al método `trade`, es decir, `self.trade(t, simbolo, ...)`. Es importante que recalcar que este método actúa como un orden a mercado con 100% de ejecución (no hay ejecuciones parciales).

Finalmente, se calculará la performance de la estrategia con 2 métricas:

- **Sharpe Ratio:** Retorno ajustado por riesgo. Usamos el promedio del PnL generado por día y volatilidad como métrica de riesgo.
- **Cvar (95%):** Indica la pérdida esperada en el 5% de los peores días. Es una métrica que muestra que tan larga es la cola de distribución PnL diarios.

Para probar su estrategia, ya habiendo implementado sus métodos, lo pueden hacer de la siguiente manera:

```
start = datetime(2023, 3, 28)
end = datetime(2023, 12, 29)

data = pd.read_parquet('data.parquet')

strategy = Strategy(data)
strategy.backtest(start, end)
results = strategy.performance()
```

Y luego pueden observar su `equity_curve`, usando `strategy.equity_curve`. Para visualización, pueden usar cualquier librería, pero les recomiendo la librería `plotly.express`.

Puntos Claves:

- **Evitar el look-ahead bias:** Asegúrate de que cualquier cálculo o indicador utilizado en la estrategia no depende de información futura.
- **Código Funcional:** El código debe funcionar correctamente, y las modificaciones deben estar bien implementadas en los métodos indicados.
- **Documentación:** Asegúrate de incluir comentarios que expliquen claramente la lógica de tu estrategia y el propósito de cada modificación.
- **Capital Inicial:** Partirán con un capital inicial de 500 mil dólares. Aprovechenlos y tengalos en consideración para cuanto es lo máximo que pueden operar con el capital que tienen, ya que cada vez que quiero realizar un trade, necesito definir una cantidad que sea acorde.
- **Entrega:** Deberán entregar una carpeta con los mismo 4 archivos y un `README.md` explicando la estrategia utilizada